

NEST JS

FULL GUIDE



Table Of Contents

Chapter 1 What is Backend Development?	01
Chapter 2 Why NestJS? (And Why Not Just Express?)	06
Chapter 3 Setting Up Your First Project	14
Chapter 4 Modules: Organizing Your App	19
Chapter 5 Controllers: Handling Requests	25
Chapter 6 Services & Dependency Injection	31
Chapter 7 NestJS Reference Cheat Sheet	37

Chapter 1

What is Backend Development?

A quick, no-fluff primer on what a server actually does, what an API is, and why you'd need something like NestJS. Sets the mental model before a single line of code.

Before we write a single line of NestJS, we need to make sure you have a solid mental model of what the backend actually is and what it does.

If you have never built a server before, this chapter is for you

The Two Sides of Every App

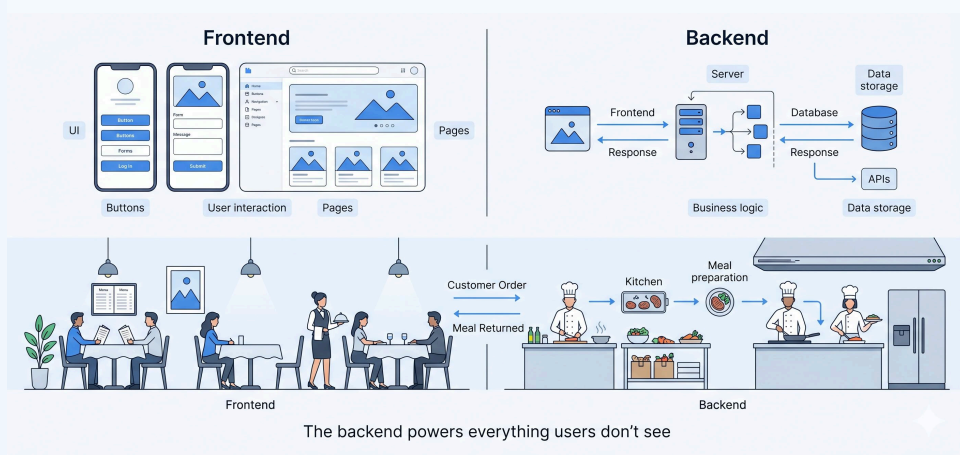
Every application you have ever used whether it is Instagram, a weather app, or an online store is split into two parts:

The **frontend** – everything you can see and interact with. Buttons, forms, images, pages.

The **backend** – everything that happens behind the scenes. Storing data, running logic, talking to databases, and serving responses.

Think of a restaurant. The frontend is the dining room, the tables, the menu, the waitstaff taking your order.

The backend is the kitchen, the chefs, the ingredients, and the actual cooking that the customer never sees.



When you log into an app and your username appears on screen, that data came from the backend.

When you post a photo and your friend sees it moments later, the backend handled that transfer.

When you buy something online and get a confirmation email, the backend sent it.

In short: The backend is where the real work happens.

What is a Server?

A server is just a program that runs continuously and listens for requests.

When someone visits your app or calls your API, they are sending a request to the server.

The server does some work reads from a database, runs some logic, checks permissions and then sends back a response.

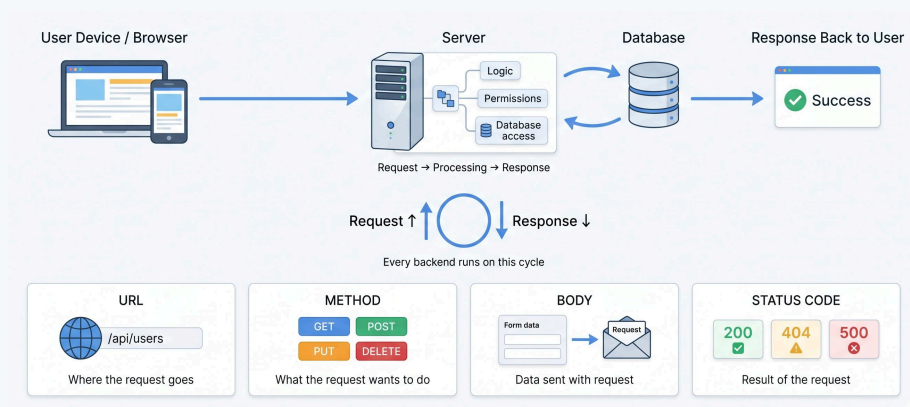
That request-response cycle is the heartbeat of every backend application.

Servers communicate using a standard called **HTTP** (HyperText Transfer Protocol).

Every time your browser loads a page or your frontend app fetches data, it sends an HTTP request.

HTTP defines the format of those requests and responses, including:

- The URL – where the request is going (e.g. /api/users)
- The method – what the request wants to do (GET, POST, PUT, DELETE)
- The body – any data sent along with the request (like a form submission)
- The status code – the result of the request (200 = OK, 404 = Not Found, 500 = Server Error)



What is an API?

You will hear the word "API" constantly as a backend developer. API stands for Application Programming Interface.

It sounds complicated, but the concept is simple.

An API is a set of rules that define how one piece of software talks to another.

In the context of web development, an API is a collection of URL endpoints that your backend exposes.

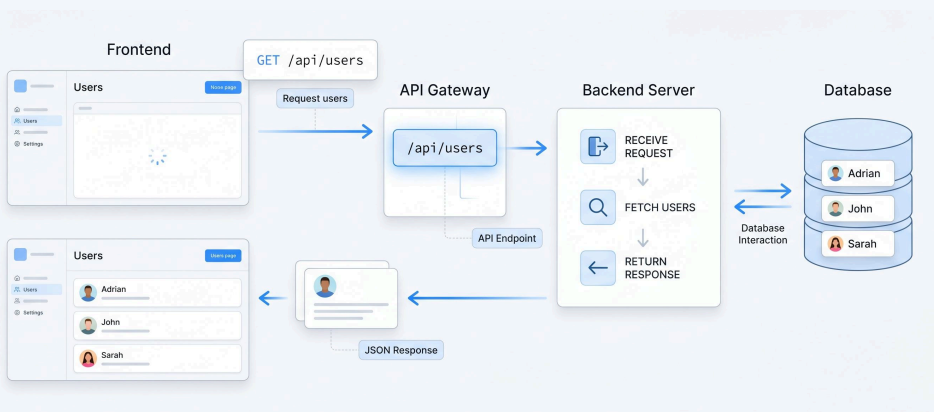
The frontend (or any other client) can call these endpoints to get data or trigger actions.

Example:

- Your frontend wants to show a list of users. It sends a GET request to **/api/users**.
- Your backend receives that request, fetches the users from the database, and sends them back as JSON.
- That URL – **/api/users** – is an API endpoint.

You are going to build many of these endpoints throughout this book.

The backend you create will be a REST API, the most common style of API on the web today.

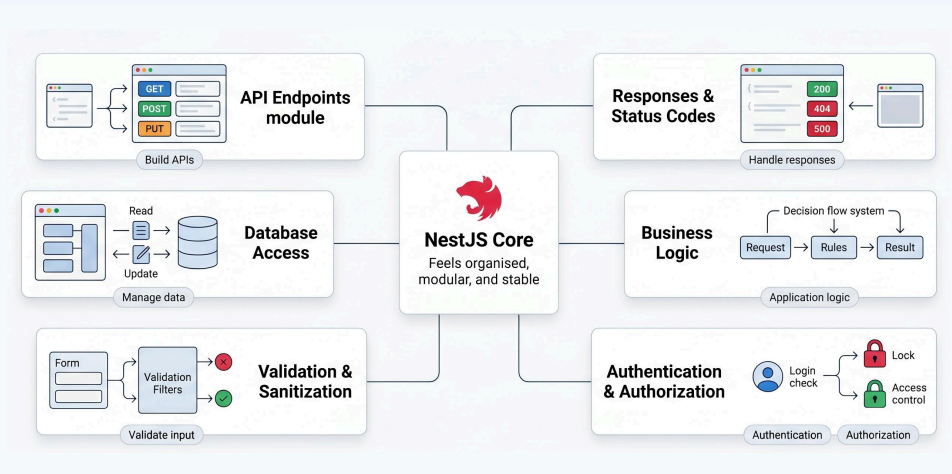


What Does a Backend Developer Actually Do?

What Does a Backend Developer Actually Do?

- Designing and building API endpoints
- Reading and writing data to a database

- Validating and sanitising incoming data
- Handling authentication (who are you?) and authorisation (what are you allowed to do?)
- Writing business logic, the rules your application operates by
- Returning the right response with the right status code



NestJS gives you a structured, professional way to do all of the above.

But before we get there, we need to understand why NestJS exists in the first place and what problem it solves.

That is exactly what Chapter 2 is about.

You now have the foundational vocabulary you need: servers, HTTP, APIs, and the request-response cycle.

Keep these concepts in mind as we move forward everything we build in this book is built on top of them.

Chapter 2

Why NestJS? (And Why Not Just Express?)

The "Wild West vs Modern City" comparison. Explain the pain of unstructured code, then show how NestJS solves it with Modules, Controllers, and Services.

Why NestJS? (And Why Not Just Express?)

Now that you understand what the backend is, you might be wondering: why do we need a framework at all?

Can't we just write a JavaScript server from scratch?

The answer is yes, you can and many developers do, using a minimal framework called Express.

So let's start there.

Express: The Wild West of Backend Development

Express.js is the most popular Node.js framework in the world. It is small, fast, and famously flexible.

When you use Express, it hands you an empty canvas and says: build whatever you want.

Here is what a simple Express server looks like:

```
const express = require('express');
const app = express();

app.get('/users', (req, res) => {
  res.json([ { id: 1, name: 'John Doe' } ]);
});

app.listen(3000);
```


Clean, simple, fast. For a small project or a quick prototype, this is perfect.

But here is the catch. Express gives you freedom. Total, unconstrained freedom. And freedom, in a growing codebase with multiple devs, is a problem.

The Problem With Too Much Freedom

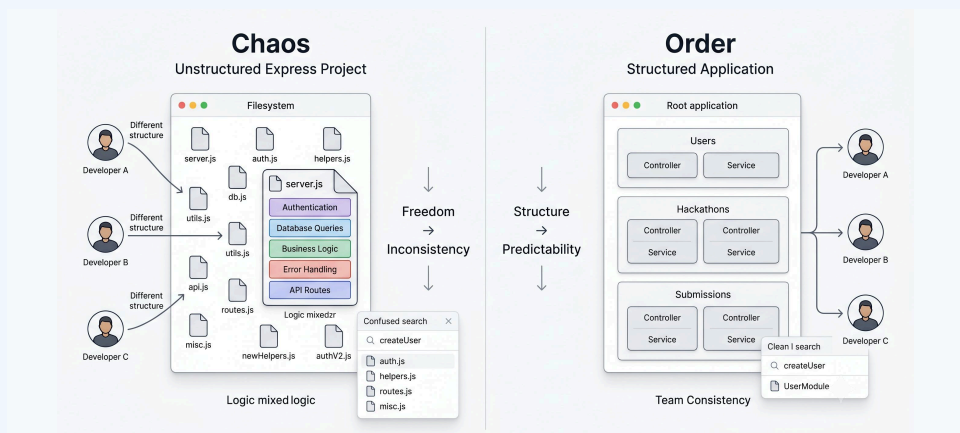
With Express, there are no rules about how you organise your code. You can:

- Write 5,000 lines of logic in a single file
- Put your database queries directly inside your route handlers
- Mix authentication logic with business logic with error handling all in one function

For solo projects, this might be fine. But the moment you join a team, or your app grows beyond a few hundred lines, things fall apart quickly.

One developer structures the code one way.

Another does it completely differently. New developers joining the project have no idea where to find anything.



The result is what developers call **spaghetti code**, a tangled mess where everything is connected to everything else, and changing one thing breaks three others.

The real cost:

In Express, you spend more time arguing about folder structure and conventions than actually building features.

NestJS: A Modern City With Zoning Laws

This is exactly the problem NestJS was built to solve.

If Express is the Wild West, NestJS is a modern city with strict zoning laws.

It does not ask you how you want to organise your code. It tells you how you must organise it and that is a feature, not a bug

NestJS is what developers call an opinionated framework.

It has strong conventions about how your application should be structured, and it enforces them.

This means you can join any NestJS project at a startup, at Netflix, at any company and know exactly where everything is on day one.

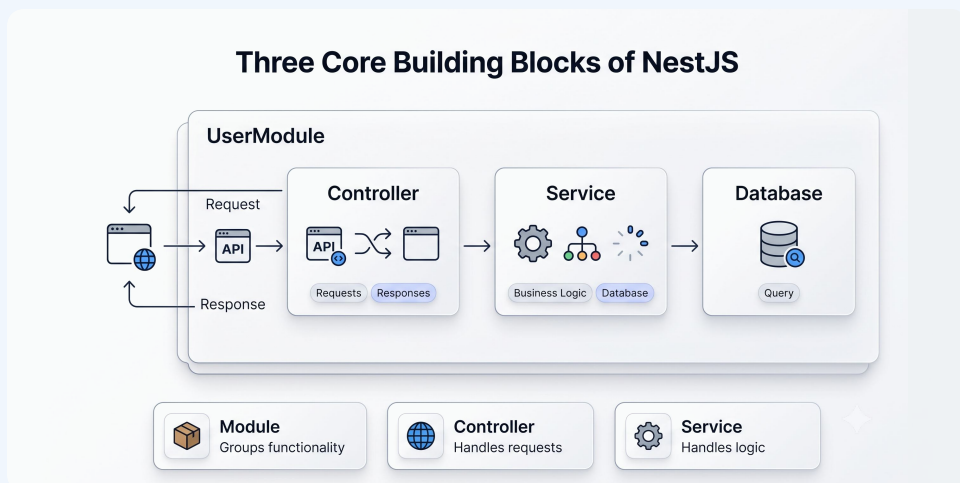
The Three Core Building Blocks

NestJS forces you to organise your application into three fundamental concepts:

- **Modules:** Containers that group related code together. Every feature of your app lives in its own module.
- **Controllers** – Classes that handle incoming HTTP requests and return responses. Think of them as the receptionist they take the request and route it to the right place.
- **Services** – Classes that contain your actual business logic. The controller asks the service to do the work; the service does it and hands the result back.

This separation of concerns is what makes NestJS applications clean and scalable. Your controllers never talk to the database directly.

Your services never worry about HTTP status codes. Everything has a single, clear responsibility.



NestJS Is Built On Top of Express

Here is something that surprises most beginners:

NestJS is not replacing Express. It is built on top of it

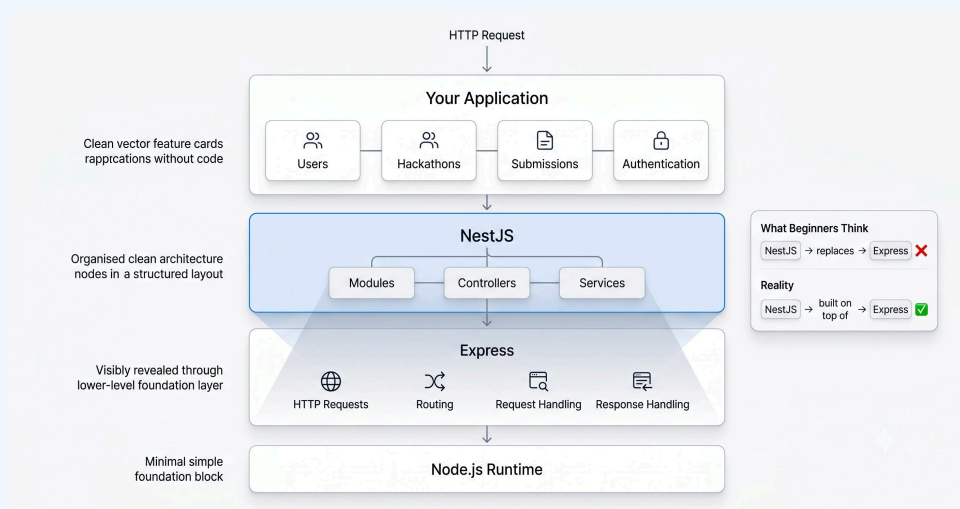
When you run a NestJS application, Express is running underneath handling the raw HTTP layer.

NestJS simply wraps it in a structured, scalable architecture. You get all the speed and reliability of Express, plus the organisation and developer experience of NestJS

Think of it this way:

NestJS is like putting a professional kitchen management system into a restaurant that already has solid plumbing and equipment.

The foundation is still Express but now everything runs to a standard.



Three Superpowers You Get With NestJS

Beyond structure, NestJS gives you three things that plain Express simply does not provide out of the box:

1. TypeScript by Default

NestJS is built with TypeScript and designed for TypeScript. There is no extra configuration needed.

From day one, you get type safety, autocomplete, and clear error messages all of which make your code far easier to write and maintain.

2. Dependency Injection

This is a design pattern (common in languages like Java and C#) that makes managing dependencies between your classes clean and testable.

Do not worry if that sounds abstract we will explain it in detail in Chapter 6.

For now, just know it is one of the things that makes NestJS applications easy to test and scale

3. Decorators

Decorators are special labels you attach to your classes and methods. They tell NestJS how to treat them. For example:

```
@Controller('users')
export class UserController {
  @Get()
  findAll() {
    return 'This returns all users';
  }
}
```

That `@Get()` decorator tells NestJS: when a GET request hits `/users`, run this method. No manual routing files. No wiring.

Just clean, readable, declarative code.

When Should You Use NestJS?

NestJS shines when:

- You are building an API that will grow over time
- You are working in a team and need consistent conventions
- You want built-in support for TypeScript, validation, error handling, & more
- You need an architecture that scales without becoming a mess

If you are just hacking together a tiny script or a quick proof of concept, plain Express might be faster to start with.

But for anything real anything you expect to maintain, extend, or hand off to another developer NestJS is the right choice.

A Quick Summary

Before we move on, here is what you should take away from this chapter:

- Express is minimal and flexible, which is great for small things but dangerous at scale
- NestJS is opinionated, it tells you how to structure your code, and that is a strength
- NestJS runs on top of Express, so you keep the performance benefits
- The three core building blocks are Modules, Controllers, and Services
- NestJS gives you TypeScript, Dependency Injection, and Decorators out of the box

You now understand the 'why' behind NestJS. In the next chapter, we will stop talking theory and start writing code.

We will set up our development environment, install the NestJS CLI, and get our first server running.

Chapter 3

Setting Up Your First Project

Node version check, installing the Nest CLI, scaffolding a project, and understanding the file structure. Walk through `main.ts` and get the server running.

Setting Up Your First Project

Enough theory. Let's get our hands dirty and get a NestJS server running on your machine.

This chapter will take you from zero to a running server in just a few minutes.

Step 1: Check Your Node.js Version

NestJS requires Node.js to run. If you haven't installed it yet, head to the official website and download it:

Download Node.js → <https://nodejs.org/en/download>

Once installed, open your terminal and run:

```
node -v
```

You should see something like v20.x.x or higher. NestJS requires Node 20+, and the current LTS (Node 24) is recommended.

If your version is lower, update it before continuing.

While you're at it, confirm npm is also available:

```
npm -v
```

You should see a version number. If both commands return a version, you're ready to go.

Step 2: Install the NestJS CLI

The NestJS CLI (Command Line Interface) is one of the best things about the framework. It automates creating projects, generating files, and keeping your architecture consistent. You will use it constantly.

Install it globally so it's available anywhere on your machine:

```
npm install -g @nestjs/cli
```

Once that finishes, verify it installed correctly:

```
nest --version
```

You should see the CLI version printed in your terminal. If you do, you're set.

Step 3: Scaffold a New Project

Creating a new NestJS project is a single command:

```
nest new my-nest-app
```

The CLI will ask you which package manager you want to use:

```
? Which package manager would you like to use?  
> npm yarn pnpm
```

Choose npm (or your preference) and hit Enter. The CLI will scaffold the entire project structure, install all dependencies, and have everything ready to go.

Step 4: Understanding the File Structure

Open the project in your code editor. You will see a structure like this:

```
my-nest-app/  
  src/  
    app.controller.ts  
    app.controller.spec.ts  
    app.module.ts  
    app.service.ts  
    main.ts  
  test/  
  node_modules/  
  package.json  
  tsconfig.json
```

Let's walk through what each file does:

- **main.ts** – The entry point of your app. This is where NestJS boots up.
- **app.module.ts** – The root module. Every NestJS app has one, and it's where everything connects.
- **app.controller.ts** – A sample controller that handles the root route (/).
- **app.service.ts** – A sample service that contains a basic piece of logic.
- **app.controller.spec.ts** – A test file for the controller. We won't focus on testing in this book, but it's there when you need it.

Don't be intimidated by this structure. By the end of this book, you will know exactly why every file is here and what it does.

Step 5: Understanding main.ts

Open main.ts. It's the first file NestJS reads when your server starts. Here's what you'll see:

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}

bootstrap();
```

This file does three things:

Don't be intimidated by this structure. By the end of this book, you will know exactly why every file is here and what it does.

1. It imports the root module (**AppModule**), which represents your entire application.
2. It uses **NestFactory.create()** to bootstrap the application – wiring everything together.
3. It starts the HTTP server, listening on port 3000 by default

Good to know:

The PORT environment variable lets you configure the port at runtime. When deployed to a platform like Heroku or Railway, the port is usually provided via environment variables

Step 6: Run the Server

Open `package.json` and you'll find several scripts. The one you'll use most during development is:

```
"start:dev": "nest start --watch"
```

The `--watch` flag enables watch mode, which means every time you save a file, the server automatically restarts. This makes development fast.

Run it:

```
npm run start:dev
```

You will see a series of green log messages in the terminal. Once you see the server is listening, open your browser and visit:

```
http://localhost:3000
```

You should see Hello World! – which comes from the default controller that NestJS generated for us.

Your server is running. That's it.

You now have a fully working NestJS server running locally. In the next chapter, we'll dig into the most fundamental building block of any NestJS application: the Module.

Chapter 4

Modules: Organizing Your App

What a module is, the building-blocks analogy, and how the module tree works. Shows how to create one with the CLI and why NestJS needs it registered to know it exists.

Modules: Organizing Your App

Now that your server is running, it's time to understand the architecture that makes NestJS so powerful.

Everything in NestJS revolves around one concept: the Module

What is a Module?

A module is a way to group related code together. Think of your application like a large corporate building.

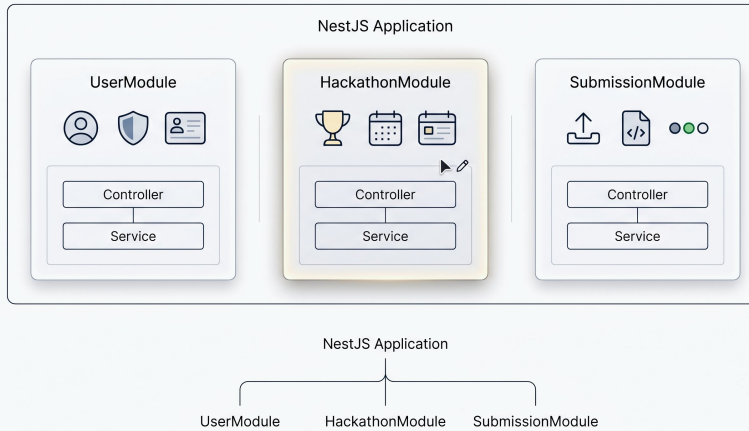
It doesn't have one big open floor where everyone shouts at each other. It has separate rooms for each department – Finance, HR, Marketing, Engineering.

In NestJS, a module is a room. Each feature of your application lives in its own module, with its own controllers, services, and logic. Nothing leaks in from the outside unless you explicitly allow it.

For example, if you were building a hackathon platform, your modules might look like this:

- **UserModule** – authentication, profiles, user data
- **HackathonModule** – creating, updating, and listing hackathons
- **SubmissionModule** – handling project submissions and tracking status

Each feature is self-contained. If you need to change how hackathons work, you go to the HackathonModule. You don't accidentally break something in the SubmissionModule.



The Root Module

Every NestJS application has one special module called the root module – by convention this is **AppModule**. This is where NestJS starts when your application boots up.

Open **app.module.ts** and you'll see:

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';

@Module({
  imports: [],
  controllers: [AppController],
  providers: [AppService]},
})

export class AppModule {}
```


Let's break down what's happening here.

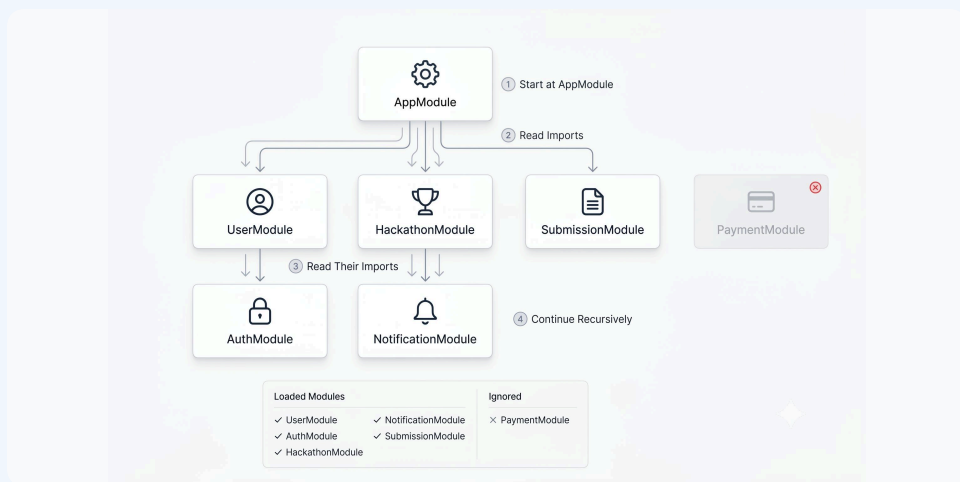
- **@Module()** – A decorator that tells NestJS this class is a module and provides its configuration.
- **imports** – Other modules whose exported providers this module needs.
- **controllers** – The controllers that belong to this module.
- **providers** – The services (and other injectables) that belong to this module.

The Module Tree

NestJS does not care about your folder structure or file names. It only knows about things through modules.

When your app starts, NestJS builds a module tree:

- It starts at the root module
- It looks at all imported modules
- Then looks at their imports, and so on recursively



If a module isn't in this tree either as the root or imported by something in the tree, NestJS does not know it exists. It won't load it, and its controllers won't respond to any requests. This is by design.

Key rule:

If NestJS doesn't know about it through the module tree, it doesn't exist. Always make sure your modules are imported somewhere.

Creating a Module With the CLI

Let's create a module for users. Open your terminal and run:

```
nest generate module user
```

Or the shorthand:

```
nest g module user
```

The CLI will:

- Create a **user/** folder inside **src/**
- Create **user.module.ts** inside it
- Automatically import **UserModule** into **AppModule** for you

Open **app.module.ts** and you'll see the import was added automatically:

```
import { UserModule } from './user/user.module';

@Module({
  imports: [UserModule], // <-- added automatically
  controllers: [AppController],
  providers: [AppService],
})

export class AppModule {}
```

This is one of the biggest time-savers in NestJS. The CLI handles the wiring.

Adding a Controller and Service

A module on its own is just an empty room. Let's add some furniture. Run these two commands:

```
nest g controller user
nest g service user
```

Open `user.module.ts` and you'll see both were automatically registered:

```
import { Module } from '@nestjs/common';
import { UserController } from './user.controller';
import { UserService } from './user.service';

@Module({
  controllers: [UserController],
  providers: [UserService],
})

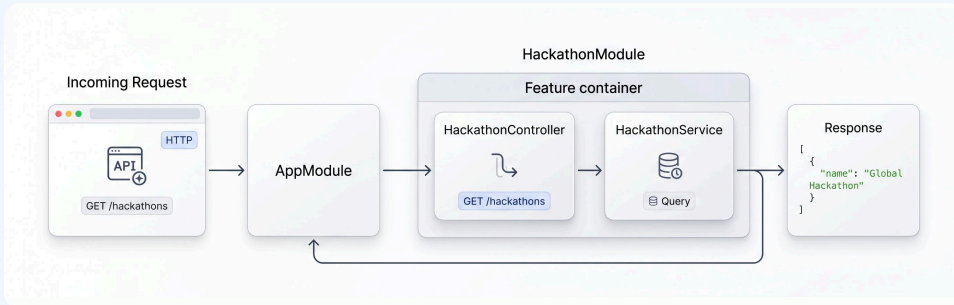
export class UserModule {}
```

The module now knows about its controller and its service. NestJS knows about the module through the root module. The full chain is connected.

The Request Flow Through a Module

Here is the mental model you should have. When a request comes in to your server:

- NestJS checks the root module for a matching route
- It follows imports to find the right feature module
- It finds the controller inside that module that handles the route
- The controller delegates to a service for the actual work
- The service returns data back up through the controller to the client



The module is the glue that makes this chain work. It's not just a folder, it's the dependency boundary that tells NestJS what belongs together.

You now understand modules: what they are, why they exist, and how NestJS uses them to build a module tree.

In the next chapter, we'll go deeper into controllers the layer that handles your HTTP requests

Chapter 5

Controllers: Handling Requests

Route method decorators (@Get, @Post, etc.), parameter decorators (@Param, @Body, @Query), and the concept of keeping controllers "thin." No heavy logic here.

Controllers: Handling Requests

A controller is the first point of contact for any incoming request. It listens for a specific URL and HTTP method, reads any data from the request, and decides what to do next.

Think of it as a traffic cop it directs traffic, but it doesn't do the heavy lifting itself.

The @Controller Decorator

Open `user.controller.ts`. The first thing you'll see is:

```
import { Controller } from '@nestjs/common';

@Controller('user')
export class UserController {}
```

The string 'user' passed to `@Controller()` sets the route prefix for this entire controller.

Every endpoint you define inside this class will automatically start with /user.

Route Method Decorators

Inside a controller, you define methods and decorate them to tell NestJS which HTTP method and path they respond to.

These are called route method decorators.

Here's a full example showing all the common ones:

```
import { Controller, Get, Post, Put, Delete } from '@nestjs/
common';

@Controller('user')
export class UserController {

  // Responds to: GET /user
  @Get() findAll() { return []; }

  // Responds to: GET /user/:id
  @Get('/:id') findOne() { return {}; }

  // Responds to: POST /user
  @Post() create() { return {}; }

  // Responds to: PUT /user/:id
  @Put('/:id') update() { return {}; }

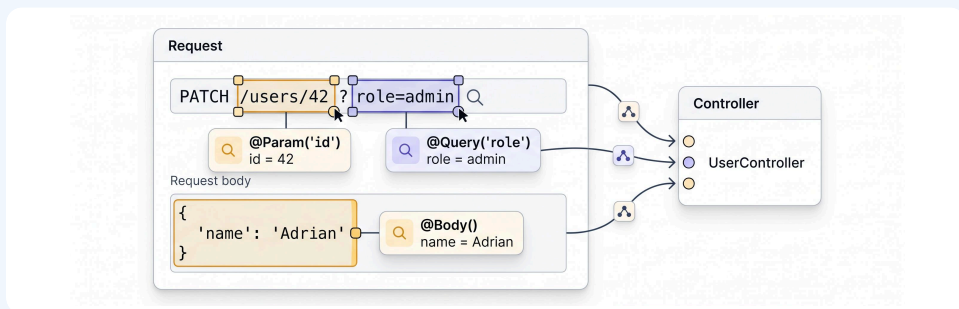
  // Responds to: DELETE /user/:id
  @Delete('/:id') remove() { return {}; }
}
```

Whatever you return from these methods, NestJS automatically serialises it as JSON and sends it as an HTTP response.

You don't need to call `res.json()` or anything like that – just return the data.

Parameter Decorators

Returning hardcoded data isn't very useful. In real endpoints, you need to read values from the request things like URL parameters, the request body, and query strings. NestJS provides a set of parameter decorators for exactly this.



Reading URL Parameters with @Param

When your route includes a dynamic segment like `:id`, use `@Param()` to read it:

```
import { Controller, Get, Param } from '@nestjs/common';

@Controller('user')
export class UserController {
  @Get('/:id')
  findOne(@Param('id') id: string) {
    return { id }; // returns { id: '42' } for GET /user/42
  }
}
```

Note:

URL params always come in as strings, even if the value looks like a number. We'll cover automatic type conversion in Chapter 8 with Pipes.

Reading the Request Body with @Body

For POST and PUT requests, the client sends data in the request body. Use `@Body()` to access it:

```
import { Controller, Post, Body } from '@nestjs/common';

@Controller('user') export class UserController {
  @Post() create(@Body() body: { name: string; email: string })
  { console.log(body.name); // 'John Doe'
    console.log(body.email); // 'john@example.com'
    return body;
  }
}
```

Reading Query Strings with @Query

Query strings are the `?key=value` parts of a URL.

Use `@Query()` to read them:

```
import { Controller, Get, Query } from '@nestjs/common';

@Controller('user')
export class UserController {
  // GET /user?name=John
  @Get()
  findAll(@Query('name') name: string) {
    return `Searching for user: ${name}`;
  }
}
```

Combining Decorators

You can combine multiple parameter decorators in a single method.

This is very common for update endpoints where you need both a URL param and a body:

```
import { Controller, Put, Param, Body } from '@nestjs/common';

@Controller('user')
export class UserController {
  // PUT /user/42 (with JSON body)
  @Put('/:id')
  update(
    @Param('id') id: string,
    @Body() body: { name: string },
  ) {
    return {
      id,
      updatedName: body.name,
    };
  }
}
```

Keep Controllers Thin

This is one of the most important principles in NestJS, so let's say it clearly:

The rule:

Controllers should only handle the request and return the response. They should not contain business logic, database calls, or complex processing.

If your controller method is more than a few lines long, that's a sign that logic belongs in a service.

Controllers are the receptionist they greet the request, check what it needs, and pass it to the right person. They don't do the work themselves.

Here's what a well-designed controller looks like after the logic is moved to a service:

```
@Controller('user')
export class UserController {
  constructor(private readonly userService: UserService) {}

  @Get()
  findAll(@Query('name') name?: string) {
    return this.userService.findAll(name);
  }

  @Post()
  create(@Body() body: CreateUserDto) {
    return this.userService.create(body);
  }
}
```

Clean, readable, and easy to test. The controller knows nothing about how users are fetched or created. It just knows who to ask.

We'll explore services and this pattern in depth in the very next chapter.

You now know how to define endpoints, read params, bodies, and query strings, and why controllers should stay thin. Next up: Services and Dependency Injection where the real logic lives.

Chapter 6

Services & Dependency Injection

What a service is and why it exists, the Pizza Chef analogy for Dependency Injection, and how `@Injectable()` + constructor injection keeps your code loosely coupled.

Services & Dependency Injection

You've seen that controllers should stay thin. So where does the real logic go? Into services.

In this chapter we'll build a service, understand why NestJS manages it for us, and learn the concept that makes it all work: Dependency Injection.

What is a Service?

A service is a class that contains your business logic. Things like:

- Fetching users from a database
- Calculating a total price for an order
- Sending an email confirmation
- Validating whether a hackathon submission is on time

Services are reusable. A single `UserService` can be used by your `UserController`, your `AuthController`, and an `AdminController`. You write the logic once and inject it wherever it's needed.

In NestJS, services are a type of `Provider` which is the general term for any class that NestJS can inject.

Every service is a provider, but not all providers have to be services. You could also have repositories, factories, or helpers as providers.

The @Injectable Decorator

Open `user.service.ts`. At the top you'll see:

```
import { Injectable } from '@nestjs/common';

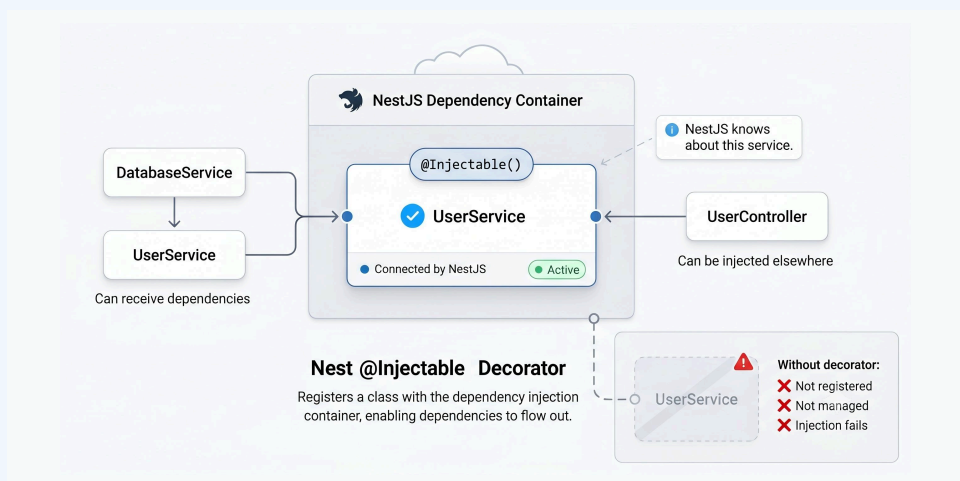
@Injectable()
export class UserService {}
```

That `@Injectable()` decorator is the key. It tells NestJS: 'this class is managed by the dependency injection system'.

This does two important things:

- It allows other dependencies to be injected into this class via its constructor
- It allows this class to be injected into other classes (like controllers)

Always add `@Injectable()` to your service classes. Without it, NestJS cannot inject other services into it, and your app will throw an error



Dependency Injection Explained

Dependency Injection (DI) is a design pattern that sounds complex but is surprisingly simple once you see the right analogy. Let's use one.

The Pizza Chef Analogy

Imagine a pizza chef who needs tomatoes. Without DI, the chef does this:

- Pauses mid-recipe
- Drives to Joe's Farm personally
- Picks the tomatoes
- Drives back
- Then continues cooking

The chef is tightly coupled to Joe's Farm. If Joe's Farm burns down, the chef can't cook. If you want to test the recipe with organic tomatoes from a different supplier, you can't the chef only knows the route to Joe's.

With Dependency Injection, the rules change:

The new rule:

The chef stays in the kitchen. When they need tomatoes, a supplier delivers them. The chef doesn't care where the tomatoes come from just that they arrive.

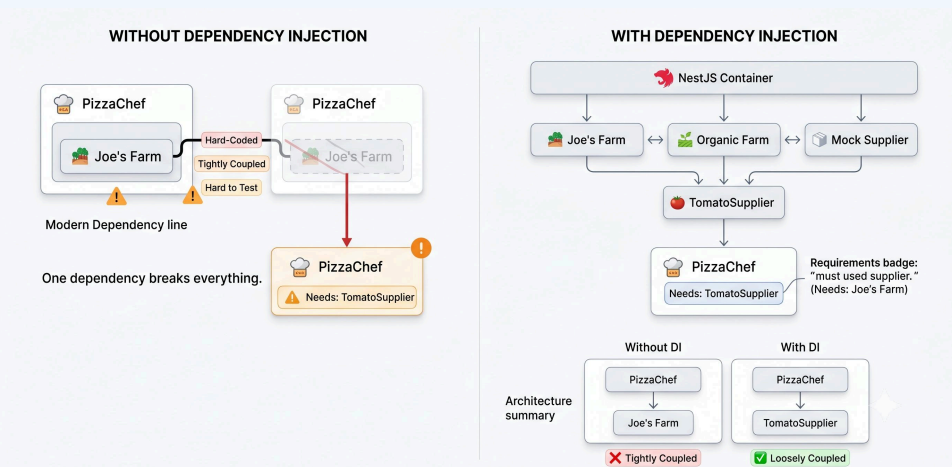
In code terms, this means the chef (your class) doesn't create its own dependencies.

It declares what it needs, and something else (NestJS) provides it:

```
// WITHOUT DI: tightly coupled
class PizzaChef {
  makePizza() {
    const farm = new JoesFarm(); // hard-coded dependency
    const tomatoes = farm.getTomatoes();
  }
}

// WITH DI: loosely coupled
class PizzaChef {
  constructor(private supplier: TomatoSupplier) {}
  makePizza() {
    // NestJS provides this
    const tomatoes = this.supplier.getTomatoes();
  }
}
```

The DI version is flexible. You can swap in any `TomatoSupplier`, Joe's Farm, an organic supplier, or a fake mock supplier for testing without changing the chef at all.



Injecting a Service Into a Controller

Now let's see this in practice. We want to inject our `UserService` into our `UserController`. Here's how:

```
import { Controller, Get } from '@nestjs/common';
import { UserService } from '../user.service';

@Controller('user')
export class UserController {
  constructor(private readonly userService: UserService) {}

  @Get()
  findAll() {
    return this.userService.findAll();
  }
}
```

That one line in the constructor does three things at once:

- Declares a class property called `userService`
- Marks it as private (only accessible inside this class)
- Tells NestJS to inject an instance of `UserService` here automatically

You don't call `new UserService()` yourself. You never manage the lifecycle of the service. NestJS handles all of that.

By default, it creates one shared instance of each service and reuses it everywhere this is called singleton scope.

Why Not Just Use new UserService()?

It might be tempting to skip DI and just create instances manually:

```
// Don't do this
findAll() {
  const userService = new UserService();
  return userService.findAll();
}
```

This works in the short term but causes real problems:

- A new instance is created every time the method runs wasteful and inconsistent
- NestJS has no control over this instance, so it can't manage its lifecycle
- The controller is now tightly coupled to the exact UserService class no swapping, no mocking for tests
- If UserService itself needs dependencies injected, they won't be there

You now understand services, providers, and the Dependency Injection pattern.

Chapter 7

NestJS Reference Cheat Sheet

A complete reference for the decorators, request lifecycle, application lifecycle hooks, and CLI commands you'll use while building NestJS applications.

Decorators

Decorators are the building blocks of NestJS. They tell the framework what a class, method, or parameter should do.

This section groups the most commonly used decorators by category, making it easy to find the one you need.

Module & Class Decorators

@Module({})

Marks a class as a NestJS module.

@Controller('path')

Marks a class as a controller and sets route prefix.

@Injectable()

Marks a class as a provider.

@Global()

Makes exported providers globally available.

@Catch(ExceptionType)

Marks a class as an exception filter.

Route Method Decorators

@Get()

Handles GET requests.

@Post()

Handles POST requests.

@Put()

Handles PUT requests.

@Patch()

Handles PATCH requests.

@Delete()

Handles DELETE requests.

@Head()

Handles HEAD requests.

@Options()

Handles OPTIONS requests.

@All()

Handles all HTTP methods.

Parameter Decorators

@Param()

Reads route parameters.

@Body()

Reads request body.

@Body('key')

Reads a single body field.

@Query()

Reads query parameters.

@Headers()

Reads request headers.

@Ip()

Gets client IP.

@Req()

Injects Express request.

@Res()

Injects Express response.

@Next()

Injects Express next function.

@HostParam()

Reads route parameters from host-based routing.

Guards, Pipes, Interceptors & Filters

@UseGuards()

Applies guards.

@UseInterceptors()

Applies interceptors.

@UsePipes()

Applies pipes.

@UseFilters()

Applies exception filters.

Response & Metadata Decorators

@HttpCode()

Overrides response status code.

@Header()

Sets custom response header.

@Redirect()

Redirects response.

@SetMetadata()

Attaches custom metadata.

Custom Decorators

@Inject(token)

Injects a provider using a custom injection token instead of the class type.

createParamDecorator()

Creates custom parameter decorators such as @CurrentUser().

@Optional()

Marks a dependency as optional during injection.

@Dependencies()

Legacy dependency injection decorator. Rarely used today.

Validation Decorators

- **@IsString()**
- **@IsNumber()**
- **@IsBoolean()**
- **@IsEmail()**
- **@IsUrl()**
- **@IsUUID()**
- **@IsOptional()**
- **@IsNotEmpty()**
- **@IsArray()**
- **@IsEnum()**
- **@IsInt()**
- **@IsPositive()**
- **@IsDate()**
- **@IsDateString()**
- **@Min()**
- **@Max()**
- **@MinLength()**
- **@MaxLength()**
- **@Matches()**
- **@ValidateNested()**

Install:

```
npm install class-validator class-transformer
```

Enable globally:

```
app.useGlobalPipes(new ValidationPipe())
```

Swagger Decorators

@ApiTags()

Groups endpoints in Swagger UI.

@ApiOperation()

Adds endpoint descriptions.

@ApiResponse()

Documents response types.

@ApiBearerAuth()

Indicates Bearer token authentication.

@ApiProperty()

Documents DTO properties.

Swagger Decorators are used when generating OpenAPI documentation.

Install:

```
npm install @nestjs/swagger swagger-ui-express
```

The Request Lifecycle

Every request follows the same path through your NestJS application.

Understanding this lifecycle helps you know where each feature belongs, from middleware and guards to controllers, services, and exception filters.

#	Layer
1	Middleware
2	Guards
3	Interceptors (before)
4	Pipes
5	Controller
6	Service
7	Interceptors (after)
8	Exception Filter

Note

Exception filters are not a normal lifecycle step. They only run when an error is thrown by middleware, guards, pipes, controllers, services, or interceptors.

Where to Put Your Logic

Task	Layer
Logging	Middleware
Auth	Guards
Response formatting	Interceptor
Type conversion	Pipe
Validation	Pipe
Business logic	Service
Error formatting	Exception Filter

Application Lifecycle Hooks

Hook	When it runs
OnModuleInit	After a module's dependencies are initialized.
OnApplicationBootstrap	After all modules have initialized.
OnModuleDestroy	Before a module is destroyed.
BeforeApplicationShutdown	Before the application shuts down.
OnApplicationShutdown	During application shutdown.

CLI Commands

The Nest CLI generates files, scaffolds features, and keeps your project structure consistent.

This section covers the commands and flags you'll use most often while building NestJS applications.

Project Commands

```
nest new <name>
nest new <name> --skip-git
nest new <name> --package-manager npm

nest build

nest start
nest start --watch
nest start --debug --watch

nest info
nest add <package>
```

Generate Commands

```
nest g module <name>
nest g controller <name>
nest g service <name>
nest g resource <name>
nest g library <name>
nest g application <name>
```

Other Schematics

```
nest g guard <name>
nest g interceptor <name>
nest g pipe <name>
nest g filter <name>
nest g middleware <name>
nest g decorator <name>
nest g class <name>
nest g interface <name>
```

Useful Flags

```
--dry-run
--flat
--no-spec
--path <path>
--module <path>
```

Common Patterns

Scaffold a feature

```
nest g module user
nest g controller user
nest g service user

# or

nest g resource user
```

Create a guard

```
nest g guard guards/auth --flat
```

Create a global exception filter

```
nest g filter common/all-exceptions --flat
```

```
app.useGlobalFilters(new AllExceptionsFilter());
```

Generate without tests

```
nest g service user --no-spec
```

You made it.

Six chapters ago, you didn't know what a server was. Now you understand how requests travel through a NestJS application, why modules exist, what controllers should and shouldn't do, and how Dependency Injection keeps your code clean and flexible.

That's not nothing that's the mental model that takes most developers months to build.

Here's what you now know:

You know that a backend is just a server listening for requests and sending back responses.

You know why NestJS exists because freedom without structure becomes spaghetti, and structure at scale becomes a superpower.

You know how to scaffold a project, wire up modules, build controllers that stay thin, and move your logic into services that NestJS manages for you.

These aren't NestJS-specific ideas.

Separation of concerns, dependency injection, keeping your routing layer away from your business logic these are principles that transfer to Express, to Django, to Laravel, to any backend you'll ever touch.

What comes next

This ebook gave you the map. Now it's time to build something real.

We've put together a full NestJS crash course on YouTube that walks through everything in this book with live code, real endpoints, and a real project.

The Nest JS Crash Course

[Watch it now →](#)



It's free, it's on the channel, and it's the natural next step from here.

And if you want to go even deeper, into databases, authentication, deployment, and building production-grade backends from scratch we're building a full **Backend Course**.

It's built around Express.js, which means you'll learn backend fundamentals closer to the metal, without a framework doing the heavy lifting for you.

That foundation makes everything else, including NestJS, click at a completely different level.

The [waitlist is open now](#).

The Ultimate Backend Course

[Join the waitlist →](#)



The best backends aren't built by people who memorised the most syntax.

They're built by people who understood the fundamentals deeply enough to apply them anywhere. You're on that path now. Keep building. 🚀